VIGNAN'S LARA
INSTITUTE OF TECHNOLOGY & SCIENCE
(AUTONOMOUS)
Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada
Accredited by NAAC 'A+' and NBA | ISO 9001 : 2015
Vadlamudi - 522 213, Guntur District

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SQL PROJECT

on

RAILWAY SYSTEM DATABASE

Submitted in partial fulfillment of the requirement for the award of

COMPUTER SCIENCE AND ENGINEERING

Submitted by

M. Yamuna

23FE1A4336(CAI)

ABSTRACT

Project Title: Railway System Database

The **Railway System Database** is a structured MySQL-based project designed to streamline and automate railway reservation processes. It handles core operations such as **train scheduling, ticket booking, seat management, passenger records, and cancellation tracking** through a relational database model.

The system is built using key tables:

- **Trains:** Stores train details including name, source, destination, and timings.
- **Passengers**: Maintains personal details like name, email, and phone.
- **Tickets:** Links passengers to trains with fields like booking date, seat number, and ticket status.

Foreign key relationships ensure referential integrity and prevent issues like double booking or invalid references.

To improve functionality and automate workflows, the project uses:

- **Triggers:** To block duplicate seat bookings and auto-log cancellations.
- **Stored Procedures:** For booking (book_ticket), cancellation (cancel_ticket), and booking history queries.
- **Views:** To simplify common reports like
  → train_schedules_by_date,
  → active_passenger_bookings, and
  → seat_occupancy_per_train.

The system also enables **advanced data analysis**, including:

- Ranking most-booked trains
- Identifying frequent passengers
- Viewing recent cancellations and refunds

These capabilities are enabled through optimized SQL queries using JOINs, aggregations, window functions, and filtering.

Key Highlights

- Clean, normalized table structure
- Use of Triggers and Procedures for automation
- Real-time insights via dynamic Views and Queries
- Scalable design ready for future extensions

In summary, this project reflects real-world database design principles with emphasis on automation, reliability, and query efficiency—making it suitable as the backend for a modern railway reservation system.

INTRODUCTION TO PROJECT

RAILWAY SYSTEM DATABASE

Brief description

● The mini-project titled “Railway System Database” is developed as part of the second-year DBMS project, fulfilling the academic requirements of the B.Tech. (Computer Science) program.

● This project focuses on designing and implementing a database to manage various aspects of a railway reservation system, including trains, passengers, tickets, and booking operations.

● It allows users to search train schedules, book and cancel tickets, and view booking history, while ensuring data consistency through the use of SQL constraints, triggers, stored procedures, and views.

Objectives

The main objective of the Railway System Database project is to design and implement a structured database that manages key railway operations efficiently. Specifically, the goals are:

1. To create and maintain a database for trains, passengers, and ticketing information.
2. To store and retrieve train details including source, destination, departure, and arrival times.
3. To ensure proper tracking of seat availability for each train.
4. To manage the process of booking tickets with unique seat allocation.
5. To enable cancellation of booked tickets .
6. To use SQL features such as views, triggers, and stored procedures for smooth and automated data handling.

Scope

Scope of the Railway System Database Project

- Comprehensive Train Data Management

The system maintains detailed information about all trains including train name, source and destination stations, departure and arrival times. This allows easy querying and scheduling for users and administrators.

- Passenger and Ticket Information Handling

It stores personal and contact details of passengers and links them with booked tickets. This helps in maintaining a full travel history for each user, enabling features like booking history and repeated user identification.

- Booking and Cancellation Operations

The database supports robust ticket booking and cancellation functionalities. It includes automatic seat availability checks, and status updates using triggers and stored procedures.

- Accurate and Secure Transactions

Through triggers and constraints, the system prevents issues such as duplicate seat bookings, invalid passenger IDs, and overlapping reservations. This ensures data integrity and improves trust in the system.

- Scalability for Integration and Reporting

The database is structured to support future integration with a web interface or mobile app. It also includes multiple views and SQL reports such as top booked trains, active passenger bookings, and seat occupancy per train for administrative analysis and decision-making.

Analysis

Railway system database is an offline backend project designed to manage train schedules, ticket bookings, passenger details, and cancellations. This system is mainly created to fulfill the following requirements, and it includes the following features:

- A central database to store complete details of trains, passengers, tickets, and cancellations.
- SQL queries and procedures to track train availability, booking status, and schedule timings.
- Each passenger is assigned a unique ID and can view their booking records anytime.
- The system maintains data integrity through constraints, triggers, and relational keys.

Table description

- **CREATING TABLES**

These SQL statements create three tables for a railway system: Trains, Passengers, and Tickets, with primary keys and relevant fields to store train details, passenger information, and ticket bookings respectively.

Trains

```
> CREATE TABLE Trains (
    train_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    source VARCHAR(100) NOT NULL,
    destination VARCHAR(100) NOT NULL,
    departure_time DATETIME NOT NULL,
    arrival_time DATETIME NOT NULL
);
```

Output: Table Created.

```
> CREATE TABLE Passengers (
    passenger_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15) NOT NULL
);
```

Output: Table Created.

```
> CREATE TABLE Tickets (
    ticket_id INT PRIMARY KEY AUTO_INCREMENT,
    passenger_id INT NOT NULL,
    train_id INT NOT NULL,
    booking_date DATE NOT NULL,
    seat_number VARCHAR(10) NOT NULL,
    status ENUM('Booked', 'Cancelled') DEFAULT 'Booked',
    FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id),
    FOREIGN KEY (train_id) REFERENCES Trains(train_id)
);
```

Output: Table Created.

The Tickets table includes foreign keys to link each ticket with a specific train and passenger, enforcing relational integrity.

- **TRAINS TABLE**

Trains table stores information about each train, including a unique ID, name, source and destination stations, and departure/arrival times.

Each record represents one train's schedule in the system.

Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

Your SQL query has been executed successfully.

```
DESC trains;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
train_id	int(11)	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	
source	varchar(100)	NO		NULL	
destination	varchar(100)	NO		NULL	
departure_time	datetime	NO		NULL	
arrival_time	datetime	NO		NULL	

Query results operations

Print Copy to clipboard Create view

TABLE : STRUCTURE OF TRAINS TABLE

- **PASSENGERS TABLE**

Passengers Table holds personal details of passengers such as a unique ID, name, email, and phone number.

Each entry represents a registered individual who can book tickets

Your SQL query has been executed successfully.

```
DESC passengers;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
passenger_id	int(11)	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
phone	varchar(15)	NO		NULL	

Query results operations

TABLE : STRUCTURE OF PASSENGERS TABLE

- **TICKETS TABLE**

Tickets table Records ticket bookings with a unique ticket ID, references to the passenger and train, booking date, seat number, and booking status.

It maintains the relationship between passengers and their booked or cancelled train journeys.

Your SQL query has been executed successfully.

```
DESC tickets;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
ticket_id	int(11)	NO	PRI	NULL	auto_increment
passenger_id	int(11)	NO	MUL	NULL	
train_id	int(11)	NO	MUL	NULL	
booking_date	date	NO		NULL	
seat_number	varchar(10)	NO		NULL	
status	enum('Booked','Cancelled')	YES		Booked	

Query results operations

TABLE : STRUCTURE OF TICKETS TABLE

- ## Insertion of Data

These SQL INSERT statements add sample data into the Trains, Passengers, and Tickets tables. They represent two trains (Shatabdi and Duronto), two passengers (Aryan and Meera), and their corresponding ticket bookings with one booked and one cancelled status.

-- sample Insertion into trains:

```
INSERT INTO Trains VALUES
(6044, 'TVC Chennai egmore special', 'secundrabad', 'villupuram', '2025-07-08 20:39:08', '2025-07-09
14:39:08');
```

Output:1 row inserted.

```
INSERT INTO Trains VALUES
(7201, 'guntur jn secundrabad', 'guntur', 'secundrabad jn', '2025-07-01 06:25:48', '2025-07-02
13:55:48');
```

Output:1 row inserted.

-- sample Insertion into passengers:

```
INSERT INTO Passengers VALUES
(1, ‘YAMUNA’, ‘yamuna@gmail.com’, ‘6300112771’);
```

Output:1 row inserted.

```
INSERT INTO Passengers VALUES
(2, ‘bhavya’, ‘bhavya@gmail.com’, ‘1234567891’);
```

Output:1 row inserted.

--sample Insertion into tickets:

```
INSERT INTO Tickets VALUES
(1, 1, 6059, '2025-06-27', 'A1', 'Cancelled');
```

Output:1 row inserted.

```
INSERT INTO Tickets VALUES
(2, 15, 6059, '2025-06-26', 'A3', 'Booked');
```

Output:1 row inserted.

- **ROWS IN TRAINS**

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	train_id	name	source	destination	departure_time	arrival_time
Edit Copy Delete	6044	TVC chennai egmore special	secundrabad	villupuram	2025-07-08 20:39:08	2025-07-09 14:39:08
Edit Copy Delete	6059	CBE BJU special	MGR chennai CTL	secundrabad jn	2025-07-09 19:40:33	2025-07-10 08:40:33
Edit Copy Delete	6060	BJU CBE Express	secundrabad jn	MGR Chennai	2025-07-10 20:42:07	2025-07-11 10:00:00
Edit Copy Delete	7016	SC BBS special	kacheguda	srikakulam	2025-07-13 18:43:48	2025-07-14 10:00:00
Edit Copy Delete	7201	guntur jn secundrabad jn special	guntur jn	secundrabad jn	2025-07-01 06:25:48	2025-07-01 13:55:48
Edit Copy Delete	7202	SECUNDERABAD JN GUNTUR JN SPECIAL	SECUNDERABAD JN	GUNTUR JN	2025-07-13 12:50:19	2025-07-14 21:50:19
Edit Copy Delete	7238	BTT MENU SPECIAL	KURNOOL CITY	KAKINADA TOWN	2025-07-14 16:49:37	2025-07-25 07:49:37
Edit Copy Delete	7240	GUNTUR JN FESTVL SPECIAL	VISAKHAPATNAM	GUNTUR	2025-07-15 07:51:44	2025-07-16 15:51:44
Edit Copy Delete	7244	RAYAGADA GUNTUR JN SPECIAL	RAYAGADA	GUNTUR JN	2025-07-12 14:48:54	2025-07-13 04:48:54
Edit Copy Delete	7246	NS FESTIVL SPECIAL	RAICHUR	KAKINADA TOWN	2025-07-15 14:51:08	2025-07-16 08:51:08
Edit Copy Delete	7258	NARASAPUR HYDERABAD DECAN SPECIAL	NARASAPUR	HYDERABAD DECAN	2025-07-26 18:00:57	2025-07-27 04:00:57
Edit Copy Delete	12705	secundrabad jn intercity superfast	guntur	secundrabad jn	2025-07-02 15:35:15	2025-07-02 22:15:15

Console

Structure of Trains table after Insertion

- **ROWS IN PASSENGERS**

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	passenger_id	name	email	phone
Edit Copy Delete	1	YAMUNA	yamuna@gmail.com	6300112771
Edit Copy Delete	2	bhavya	bhavya@gmail.com	1234567891
Edit Copy Delete	3	geethika	geethika@gmail.com	1234567892
Edit Copy Delete	4	samrajyam	rajyam@gmail.com	1234567893
Edit Copy Delete	5	varshini	varshini@gmail.com	1234567894
Edit Copy Delete	6	kavya	kavya@gmail.com	1234567895
Edit Copy Delete	7	gowthami	gowthami@gmail.com	1234567896
Edit Copy Delete	8	manasa	manasa@gmail.com	1122334455
Edit Copy Delete	9	dhanusha	dhanusha@gmail.com	2233445566
Edit Copy Delete	10	harshitha	harshitha@gmail.com	3344556677
Edit Copy Delete	11	pavani	pavani@gmail.com	1133445566
Edit Copy Delete	12	sajida	sajida@gmail.com	2244556677
Edit Copy Delete	13	sai	sai@gmail.com	5566778899
Console lit Copy Delete	14	harika	harika@gmail.com	9322119203

Structure of Passengers table after Insertion

- **ROWS IN TICKETS**

```
SELECT * FROM tickets;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

←T→	ticket_id	passenger_id	train_id	booking_date	seat_number	status
Edit Copy Delete	1	1	6059	2025-06-27	A1	Cancelled
Edit Copy Delete	2	15	6059	2025-06-26	A2	Booked
Edit Copy Delete	3	5	6060	2025-06-30	B1	Booked
Edit Copy Delete	4	4	7016	2025-07-01	A3	
Edit Copy Delete	5	12	7016	2025-07-02	A5	Cancelled
Edit Copy Delete	6	13	7238	2025-07-03	AS1	Cancelled
Edit Copy Delete	7	11	7246	2025-07-01	A7	Cancelled
Edit Copy Delete	8	8	12705	2025-07-04	B2	Booked
Edit Copy Delete	9	6	12705	2025-07-02	B3	Cancelled
Edit Copy Delete	10	10	17201	2025-07-01	B4	Cancelled
Console lit Copy Delete	11	14	17228	2025-06-30	A6	Cancelled

Structure of Tickets table after Insertion

- **Cancellation log :**

The cancellation_log table stores records of cancelled tickets, including the associated ticket_id, passenger_id, time of cancellation, and refund amount.

It ensures each entry is uniquely identified by log_id and uses CURRENT_TIMESTAMP to automatically log when the cancellation occurs.

Table creation:

```
> CREATE TABLE IF NOT EXISTS cancellation_log (
        log_id INT AUTO_INCREMENT PRIMARY KEY,
        ticket_id INT,
        passenger_id INT,
        cancellation_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        refund_amount DECIMAL(10,2)
    );
```

Table created.

```
DESC cancellation_log;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
log_id	int(11)	NO	PRI	NULL	auto_increment
ticket_id	int(11)	YES		NULL	
passenger_id	int(11)	YES		NULL	
cancellation_time	timestamp	NO		current_timestamp()	
refund_amount	decimal(10,2)	YES		NULL	

TABLE : STRUCTURE OF CONFRIMATION_LOG TABLE

-- sample Insertion into cancellation_log:

```
INSERT INTO cancellation_log VALUES
(101, 4, 4, '2025-07-13 22:55:24', ' ');
```

Output:1 row inserted.

- **ROWS IN CANCELLATION_LOG**

```
SELECT * FROM cancellation_log;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	log_id	ticket_id	passenger_id	cancellation_time	refund_amount
Edit Copy Delete	101	4	4	2025-07-13 22:55:24	NULL
Edit Copy Delete	102	6	13	2025-07-15 09:57:43	500.00
Edit Copy Delete	103	2	15	2025-07-10 02:58:57	450.00
Edit Copy Delete	104	1	1	2025-07-07 22:00:44	599.00
Edit Copy Delete	105	3	5	2025-07-11 10:01:38	200.00

Check all *With selected:* Edit Copy Delete Export

Structure of Cancellation_log table after Insertion

○ Confrimation log :

```
> CREATE TABLE IF NOT EXISTS confirmation_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    ticket_id INT NOT NULL,
    passenger_id INT NOT NULL,
    confirmation_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (ticket_id) REFERENCES Tickets(ticket_id),
    FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id)
);
```

```
desc confirmation_log;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
log_id	int(11)	NO	PRI	NULL	auto_increment
ticket_id	int(11)	NO	MUL	NULL	
passenger_id	int(11)	NO	MUL	NULL	
confirmation_time	timestamp	NO		current_timestamp()	

Query results operations

Print Copy to clipboard Create view

TABLE : STRUCTURE OF CONFRIMATION_LOG TABLE

-- sample Insertion into confrimation_log:

```
INSERT INTO confrimation_log VALUES
(202, 4, 4, '2025-07-13 20:23:48');
```

Output:1 row inserted.

- **ROWS IN CONFRIMATION_LOG**

Structure of Confrimation_log table after Insertion

○ Triggers:

A trigger in SQL is a stored procedure that automatically executes (fires) in response to certain events on a particular table or view, such as INSERT, UPDATE, or DELETE. Triggers are used to enforce business rules, maintain data integrity, and automate system tasks within the database Triggers: A trigger in SQL is a stored procedure that automatically executes (fires) in response to certain events on a particular table or view, such as INSERT, UPDATE, or DELETE. Triggers are used to enforce business rules, maintain data integrity, and automate system tasks within the database

- **CHECK SEAT AVAILABILITY BEFORE BOOKING**

This SQL trigger check_seat_availability ensures no double-booking of seats on a train. Before inserting a new ticket, it checks if the seat is already booked for that train; if so, it raises an error and prevents the insertion.

Trigger to Check Seat Availability Before Booking-

```
> DELIMITER //
  CREATE TRIGGER check_seat_availability
  BEFORE INSERT ON Tickets
  FOR EACH ROW
  BEGIN
 DECLARE seat_taken INT;

 SELECT COUNT(*) INTO seat_taken
 FROM Tickets
 WHERE train_id = NEW.train_id
 AND seat_number = NEW.seat_number
```

```
AND status = 'Booked';

IF seat_taken > 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Seat already booked. Please choose another seat.';
END IF;
END //
DELIMITER ;
```

Output: Trigger created

- **SENDING CONFIRMATION UPON TICKET BOOKING**

The send_booking_confirmation trigger runs after a new ticket is inserted into the Tickets table.

It logs a confirmation message into the confirmation_log table with ticket and passenger details.

Trigger to Send Confirmation Upon Booking-

```
> DELIMITER //
  CREATE TRIGGER send_booking_confirmation
  AFTER INSERT ON Tickets
  FOR EACH ROW
  BEGIN
  INSERT INTO confirmation_log (ticket_id, passenger_id)
  VALUES (NEW.ticket_id, NEW.passenger_id);
  END //
  DELIMITER ;
```

Output: Trigger created

- **LOG TICKET CANCELLATION**

The log_ticket_cancellation trigger executes after a ticket's status changes from 'Booked' to 'Cancelled'.

It calculates the time left before departure, decides the refund (80% or 50%), and logs the cancellation details into the cancellation_log table.

Trigger to log ticket cancellation-

```
DELIMITER //
  CREATE TRIGGER log_ticket_cancellation
  AFTER UPDATE ON Tickets
  FOR EACH ROW
  BEGIN
  DECLARE hours_before_departure INT;1
  DECLARE refund DECIMAL(10,2);

IF OLD.status = 'Booked' AND NEW.status = 'Cancelled' THEN
-- Calculate hours before departure
SELECT TIMESTAMPDIFF(HOUR, NOW(), departure_time) INTO hours_before_departure
FROM Trains
WHERE train_id = NEW.train_id;
```

```
-- Calculate refund
 IF hours_before_departure > 24 THEN
 SET refund = 80.00; -- 80% refund
 ELSE
 SET refund = 50.00; -- 50% refund
END IF;

-- Log the cancellation
 INSERT INTO cancellation_log (ticket_id, passenger_id, refund_amount)
 VALUES (NEW.ticket_id, NEW.passenger_id, refund);
 END IF;
 END //
DELIMITER ;
```

Output: Trigger created

Checking Triggers :-

Your SQL query has been executed successfully.

```
show TRIGGERS;
```

Profiling [Edit inline] [Edit] [Create PHP code] [Refresh]

Extra options

Trigger	Event	Table	Statement	Timing	Created	sql_mode
check_seat_availability	INSERT	tickets	BEGIN DECLARE seat_taken INT; SELE...	BEFORE	2025-07-30 07:43:38.79	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBST
send_booking_confirmation	INSERT	tickets	BEGIN INSERT INTO confirmation_log (ticket_id...	AFTER	2025-07-30 07:47:55.90	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBST
log_ticket_cancellation	UPDATE	tickets	BEGIN DECLARE hours_before_departure INT;	AFTER	2025-07-30 07:52:52.38	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBST

○ Stored Procedures:

- **BOOK A NEW TICKET**

The book_ticket procedure first checks if the requested seat on a train is already booked.

If available, it inserts a new ticket with the next ticket ID; otherwise, it raises an error message.

```
> DELIMITER //
    CREATE PROCEDURE book_ticket(
    IN p_passenger_id INT,
    IN p_train_id INT,
    IN p_seat_number VARCHAR(10),
    OUT p_ticket_id INT
    )
    BEGIN
    DECLARE seat_available INT DEFAULT 0;

    -- Check if seat is available
    SELECT COUNT(*) INTO seat_available
    FROM Tickets
    WHERE train_id = p_train_id


    AND seat_number = p_seat_number
    AND status = 'Booked';

     IF seat_available > 0 THEN
     SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Seat already booked. Please choose another seat.';
     ELSE
    -- Get next ticket_id
    SELECT IFNULL(MAX(ticket_id), 0) + 1 INTO p_ticket_id FROM Tickets;

    -- Insert the ticket
    INSERT INTO Tickets (ticket_id, passenger_id, train_id, booking_date, seat_number,
status)
    VALUES (p_ticket_id, p_passenger_id, p_train_id, CURDATE(), p_seat_number, 'Booked');
    END IF;
    END //
    DELIMITER ;
```

Output: procedure created

- **VIEW PASSENGER BOOKING HISTORY**

The passenger_booking_history procedure retrieves all ticket bookings made by a specific passenger. It displays ticket details along with train information, ordered by the most recent booking date

```
> DELIMITER //
```

```
CREATE PROCEDURE passenger_booking_history(IN p_passenger_id INT)
BEGIN
SELECT
t.ticket_id,
tr.name AS train_name,
tr.source,
tr.destination,
tr.departure_time,
tr.arrival_time,
t.booking_date,
t.seat_number,
t.status
FROM Tickets t
JOIN Trains tr ON t.train_id = tr.train_id
WHERE t.passenger_id = p_passenger_id
ORDER BY t.booking_date DESC;
END //
DELIMITER ;
```

Output: procedure created

- **CANCEL A TICKET AND UPDATE SEAT STATUS**

The cancel_ticket procedure checks the current status of a ticket by its ID.
If the ticket exists and is not already cancelled, it updates the status to 'Cancelled' and confirms the action.

```
> DELIMITER //
  CREATE PROCEDURE cancel_ticket(IN p_ticket_id INT)
  BEGIN
  DECLARE current_status VARCHAR(20);

  -- Get current status
   SELECT status INTO current_status FROM Tickets WHERE ticket_id = p_ticket_id;

  IF current_status IS NULL THEN
   SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Ticket not found.';
   ELSEIF current_status = 'Cancelled' THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Ticket is already cancelled.';
  ELSE
```

```
UPDATE Tickets
SET status = 'Cancelled'
WHERE ticket_id = p_ticket_id;

SELECT 'Ticket cancelled successfully.' AS message;
END IF;
END //
DELIMITER ;
```

Output: procedure created

Checking Stored Procedures :-

○ Views:

- **TRAIN SCHEDULES BY DATE**

The train_schedules_by_date view organizes train schedule details by extracting and displaying the date and time of departures and arrivals.
It helps users quickly view all train schedules sorted by travel date and departure time.

```
> CREATE VIEW train_schedules_by_date AS
    SELECT
    train_id,
    name AS train_name,
    source,
    destination,
    DATE(departure_time) AS travel_date,
```

```
TIME(departure_time) AS departure_time,
TIME(arrival_time) AS arrival_time
FROM Trains
ORDER BY travel_date, departure_time;
```

Output:

Showing rows 0 - ... (Query took 0.0008 seconds.)

```
SELECT * FROM `train_schedules_by_date`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

> | Show all | Number of rows: 25 Filter rows: Search this table

Extra options

	train_id	train_name	source	destination	travel_date	departure_time	arrival_time
Edit Copy Delete	7201	guntur jn secundrabad jn special	guntur jn	secundrabad jn	2025-07-01	06:25:48	13:55:48
Edit Copy Delete	17239	simhadri express	guntur	visakhapatanam	2025-07-02	14:02:54	14:02:54
Edit Copy Delete	12705	secundrabad jn intercity superfast	guntur	secundrabad jn	2025-07-02	15:35:15	22:15:15
Edit Copy Delete	17243	guntur jn rayagada express	guntur	rayagada	2025-07-02	23:20:18	13:40:18
Edit Copy Delete	17201	Golconda express	guntur	secundrabad	2025-07-03	06:30:29	13:55:29
Edit Copy Delete	57305	KCGGNT PASSENGER	kacheguda	guntur jn	2025-07-03	14:46:52	08:46:52
Edit Copy Delete	12747	palanadu superfast express	guntur	vikarabad	2025-07-04	05:45:00	12:15:00
Console lit Copy Delete	17228	guntur jn dhone express	guntur jn	dhone	2025-07-05	13:00:12	21:20:12

- **ACTIVE BOOKINGS OF PASSENGERS**

The active_passenger_bookings view displays details of all passengers with currently booked tickets, including their contact info and train details.
It filters only 'Booked' tickets and sorts the output by passenger name and train departure time.

```
> CREATE VIEW active_passenger_bookings AS
        SELECT
         p.passenger_id,
        p.name AS passenger_name,
         p.email,
        p.phone,
        t.ticket_id,
        tr.name AS train_name,
         tr.source,
        tr.destination,
        tr.departure_time,
        tr.arrival_time,
        t.booking_date,
        t.seat_number
        FROM Passengers p
        JOIN Tickets t ON p.passenger_id = t.passenger_id
        JOIN Trains tr ON t.train_id = tr.train_id
        WHERE t.status = 'Booked'
        ORDER BY p.name, tr.departure_time;
```

output:

enger_id	passenger_name	email	phone	ticket_id	train_name	source	destination	departure_time	arrival_time	booking_date	seat_n
2	bhavya	bhavya@gmail.com	1234567891	15	guntur jn kachiguda passenger	guntur jn	kacheguda	2025-07-29 17:10:09	2025-07-29 09:45:09	2025-06-25	C1
7	gowthami	gowthami@gmail.com	1234567896	12	simhadri express	guntur	visakhapatanam	2025-07-02 14:02:54	2025-07-03 14:02:54	2025-06-29	A8
8	manasa	manasa@gmail.com	1122334455	8	secundrabad jn intercity superfast	guntur	secundrabad jn	2025-07-02 15:35:15	2025-07-02 22:15:15	2025-07-04	B2
25	nikitha	nikitha@gmail.com	7744332211	16	guntur jn kachiguda passenger	guntur jn	kacheguda	2025-07-29 17:10:09	2025-07-29 09:45:09	2025-07-01	A12
16	SHIVANI	shivani@gmail.com	9988776655	25	Golconda express	guntur	secundrabad	2025-07-03 06:30:29	2025-07-03 13:55:29	2025-07-10	D8
22	sonu	sonu@gmail.com	6622331144	18	KACHEGUDA GUNTUR JN EXPRESS	KACHEGUDA	GUNTUR JN	2025-07-21 21:55:46	2025-07-22 12:55:46	2025-07-04	D3
17	srihari	srihari@gmail.com	7788665544	24	GOLCONDA EXPRESS	SECUNDERRABAD JN	GUNTUR JN	2025-07-27 12:59:32	2025-07-28 21:59:32	2025-07-07	B7
19	swathi	swathi@gmail.com	9955667788	22	simhadri express	guntur	visakhapatanam	2025-07-02 14:02:54	2025-07-03 14:02:54	2025-07-08	D5
23	teja	teja@gmail.com	8833224455	17	DHONE GUNTUR JN EXPRESS	DHONE	GUNTUR JN	2025-07-30 06:54:45	2025-07-30 14:54:45	2025-07-01	AB4

Console

- **SEAT OCCUPANCY PER TRAIN**

The seat_occupancy_per_train view summarizes ticket status for each train, showing how many seats are booked, cancelled, and the total issued.

It groups data by train and helps monitor seat occupancy trends across all trains.

```
> CREATE VIEW seat_occupancy_per_train AS
  SELECT
   t.train_id,
  tr.name AS train_name,
  tr.source,
  tr.destination,
  COUNT(CASE WHEN t.status = 'Booked' THEN 1 END) AS booked_seats,
  COUNT(CASE WHEN t.status = 'Cancelled' THEN 1 END) AS cancelled_seats,
  COUNT(*) AS total_tickets
  FROM Tickets t
  JOIN Trains tr ON t.train_id = tr.train_id
  GROUP BY t.train_id, tr.name, tr.source, tr.destination;
```

output:

```
SELECT * FROM `seat_occupancy_per_train`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

train_id	train_name	source	destination	booked_seats	cancelled_seats	total_tickets
6059	CBE BJU special	MGR chennai CTL	secundrabad jn	1	1	2
6060	BJU CBE Express	secundrabad jn	MGR Chennai	1	0	1
7016	SC BBS special	kacheguda	srikakulam	0	1	2
7238	BTT MENU SPECIAL	KURNOOL CITY	KAKINADA TOWN	0	1	1
7246	NS FESTIVL SPECIAL	RAICHUR	KAKINADA TOWN	0	1	1
12705	secundrabad jn intercity superfast	guntur	secundrabad jn	1	1	2
17201	Golconda express	guntur	secundrabad	1	1	2
17202	GOLCONDA EXPRESS	SECUNDERRABAD JN	GUNTUR JN	1	0	1
17228	guntur jn dhone express	guntur jn	dhone	0	2	2
17239	simhadri express	guntur	visakhapatanam	2	0	2
17243	guntur jn rayagada express	guntur	rayagada	0	2	2
17251	guntur jn kacheguda express	guntur jn	kachegunda	0	2	2
17252	KACHEGUDA GUNTUR JN EXPRESS	KACHEGUDA	GUNTUR JN	1	1	2
17277	DHONE GUNTUR JN EXPRESS	DHONE	GUNTUR JN	1	0	1
57306	guntur jn kachiguda passenger	guntur jn	kacheguda	2	0	2

Console

○ Nested Queries:

- **TRAINS WITH MAXIMUM TICKETS BOOKED**

This query identifies the train(s) with the highest number of currently booked tickets. It uses the RANK() window function to rank trains by ticket count and filters only those with the top rank (rank_val = 1).

```
> SELECT  train_id,name AS train_name, source, destination, ticket_count FROM (SELECT
  t.train_id,tr.name, tr.source,tr.destination COUNT(*) AS ticket_count, RANK() OVER (ORDER BY
  COUNT(*)   DESC) AS rank_val FROM Tickets t JOIN Trains tr ON t.train_id = tr.train_id
  WHERE t.status = 'Booked'
  GROUP BY t.train_id, tr.name, tr.source, tr.destination
  ) AS ranked_trains
  WHERE rank_val = 1;
```

OUTPUT:

Showing rows 0 - 1 (2 total, Query took 0.0010 seconds.)

```
SELECT train_id, name AS train_name, source, destination, ticket_count FROM ( SELECT t.train_id, tr.name, tr.source, tr.destination,
COUNT(*) AS ticket_count, RANK() OVER (ORDER BY COUNT(*) DESC) AS rank_val FROM Tickets t JOIN Trains tr ON t.train_id = tr.train_id
WHERE t.status = 'Booked' GROUP BY t.train_id, tr.name, tr.source, tr.destination ) AS ranked_trains WHERE rank_val = 1;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

train_id	train_name	source	destination	ticket_count
17239	simhadri express	guntur	visakhapatanam	2
57306	guntur jn kachiguda passenger	guntur jn	kacheguda	2

Show all | Number of rows: 25 Filter rows: Search this table

- **PASSENGERS WITH MORE THAN 3 BOOKINGS**

This SQL query retrieves the details of passengers who have more than 3 active (Booked) tickets.

It joins the Passengers table with a subquery that counts bookings per passenger and filters for those with a count greater than 3.

```
> SELECT
    p.passenger_id,p.name AS passenger_name,p.email,booking_count FROM Passengers p
    JOIN (
    SELECT passenger_id,
    COUNT(*) AS booking_count
    FROM Tickets
    WHERE status = 'Booked'
    GROUP BY passenger_id
    HAVING COUNT(*) > 3
    ) AS passenger_bookings ON p.passenger_id = passenger_bookings.passenger_id;
```

OUTPUT:

OUTPUT: NO PASSENGER WITH MORE THAN 3 BOOKING

- **TICKETS CANCELLED IN THE LAST MONTH**

This SQL query retrieves all tickets that were cancelled in the last 1 month, showing ticket details, passenger and train names, booking and cancellation dates, seat numbers, and refund amounts. It joins the Tickets, Passengers, Trains, and cancellation_log tables and filters based on recent cancellation time and ticket status.

```
> SELECT t.ticket_id,p.name AS passenger_name,tr.name AS train_name,t.booking_date,
    t.seat_number,cl.cancellation_time, cl.refund_amount
FROM Tickets t
JOIN Passengers p ON t.passenger_id = p.passenger_id
JOIN Trains tr ON t.train_id = tr.train_id
JOIN cancellation_log cl ON t.ticket_id = cl.ticket_id WHERE t.status = 'Cancelled'
AND cl.cancellation_time >= DATE_SUB(CURRENT_DATE(), INTERVAL 1 MONTH);
```

OUTPUT:

Showing rows 0 - 1 (2 total, Query took 0.0010 seconds.)

```
SELECT t.ticket_id, p.name AS passenger_name, tr.name AS train_name, t.booking_date, t.seat_number, cl.cancellation_time,
cl.refund_amount FROM Tickets t JOIN Passengers p ON t.passenger_id = p.passenger_id JOIN Trains tr ON t.train_id = tr.train_id JOIN
cancellation_log cl ON t.ticket_id = cl.ticket_id WHERE t.status = 'Cancelled' AND cl.cancellation_time >= DATE_SUB(CURRENT_DATE(),
INTERVAL 1 MONTH);
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

ticket_id	passenger_name	train_name	booking_date	seat_number	cancellation_time	refund_amount
6	sai	BTT MENU SPECIAL	2025-07-03	AS1	2025-07-15 09:57:43	500.00
1	YAMUNA	CBE BJU special	2025-06-27	A1	2025-07-07 22:00:44	599.00

○ Join-Based Queries:

- **TICKET DETAILS WITH PASSENGER AND TRAIN INFO**

This SQL query retrieves a complete list of ticket bookings, combining ticket, passenger, and train details such as passenger name, contact, train info, seat number, and booking status. It joins the Tickets, Passengers, and Trains tables and orders the results by the most recent booking dates first.

```
> SELECT t.ticket_id,p.name AS passenger_name,p.email,p.phone,
  tr.name AS train_name,tr.source,tr.destination,tr.departure_time,tr.arrival_time,
  t.booking_date,t.seat_number,t.status
  FROM Tickets t
  JOIN Passengers p ON t.passenger_id = p.passenger_id
  JOIN Trains tr ON t.train_id = tr.train_id
  ORDER BY t.booking_date DESC;
```

output:

ticket_id	passenger_name	email	phone	train_name	source	destination	departure_time	arrival_time
31	varshini	varshini@gmail.com	1234567894	CBE BJU special	MGR chennai CTL	secundrabad jn	2025-07-09 19:40:33	2025-07-10 08:4
20	murali	murali@gmail.com	2255667788	guntur jn kacheguda express	guntur jn	kachegunda	2025-07-06 17:30:19	2025-07-07 07:2
25	SHIVANI	shivani@gmail.com	9988776655	Golconda express	guntur	secundrabad	2025-07-03 06:30:29	2025-07-03 13:5
22	swathi	swathi@gmail.com	9955667788	simhadri express	guntur	visakhapatanam	2025-07-02 14:02:54	2025-07-03 14:0
21	swathi	swathi@gmail.com	9955667788	guntur jn rayagada express	guntur	rayagada	2025-07-02 23:20:18	2025-07-03 13:4
24	srihari	srihari@gmail.com	7788665544	GOLCONDA EXPRESS	SECUNDERRABAD JN	GUNTUR JN	2025-07-27 12:59:32	2025-07-28 21:5
19	karthik	karthik@gmail.com	5511223344	KACHEGUDA GUNTUR JN EXPRESS	KACHEGUDA	GUNTUR JN	2025-07-21 21:55:46	2025-07-22 12:5
8	manasa	manasa@gmail.com	1122334455	secundrabad jn intercity superfast	guntur	secundrabad jn	2025-07-02 15:35:15	2025-07-02 22:1
				KACHEGUDA				

Console

- **TRAINS DEPARTING FROM A SPECIFIC STATION**

This SQL query retrieves details of all trains departing from Delhi, including their name, source, destination, and timing.

The results are ordered by the departure time, showing the earliest trains first.

```
> SELECT train_id,name AS train_name,source,destination,departure_time,arrival_time
  FROM Trains
  WHERE source = 'Delhi' -- Replace with parameter if needed
  ORDER BY departure_time;
```

output:

Showing rows 0 - 4 (5 total, Query took 0.0007 seconds.) [departure_time: **2025-07-02 14:02:54... - 2025-07-04 05:45:00...**]

```
SELECT train_id, name AS train_name, source, destination, departure_time, arrival_time FROM Trains WHERE source = 'guntur' -- Replace
with parameter if needed ORDER BY departure_time;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

←T→	train_id	train_name	source	destination	departure_time	arrival_time
Edit Copy Delete	17239	simhadri express	guntur	visakhapatanam	2025-07-02 14:02:54	2025-07-03 14:02:54
Edit Copy Delete	12705	secundrabad jn intercity superfast	guntur	secundrabad jn	2025-07-02 15:35:15	2025-07-02 22:15:15
Edit Copy Delete	17243	guntur jn rayagada express	guntur	rayagada	2025-07-02 23:20:18	2025-07-03 13:40:18
Edit Copy Delete	17201	Golconda express	guntur	secundrabad	2025-07-03 06:30:29	2025-07-03 13:55:29
Edit Copy Delete	12747	palanadu superfast express	guntur	vikarabad	2025-07-04 05:45:00	2025-07-04 12:15:00

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

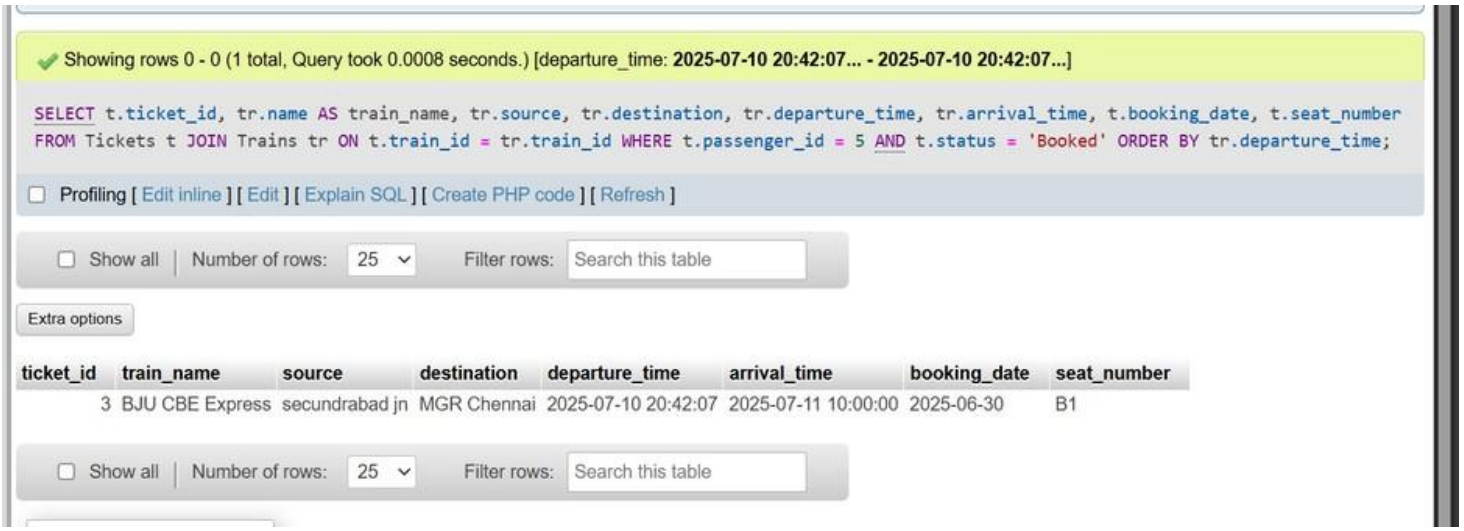
- **BOOKED TICKETS FOR A SPECIFIC PASSENGER**

This SQL query retrieves all booked tickets for a specific passenger (with passenger_id = 5).

It displays ticket details along with corresponding train information, sorted by the train's departure time.

```
> SELECT t.ticket_id,tr.name AS train_name,tr.source,tr.destination,tr.departure_time,
  tr.arrival_time,t.booking_date,t.seat_number FROM Tickets t JOIN Trains tr ON t.train_id =
  tr.train_id WHERE t.passenger_id = 5 AND t.status = 'Booked' ORDER BY tr.departure_time;
```

output:



DESIGN

ER Diagram

An entity–relationship model describes inter-related things of interest in a specific domain of knowledge. An ER model is composed of entity types and specifies relationships that can exist between instances of those entity types.

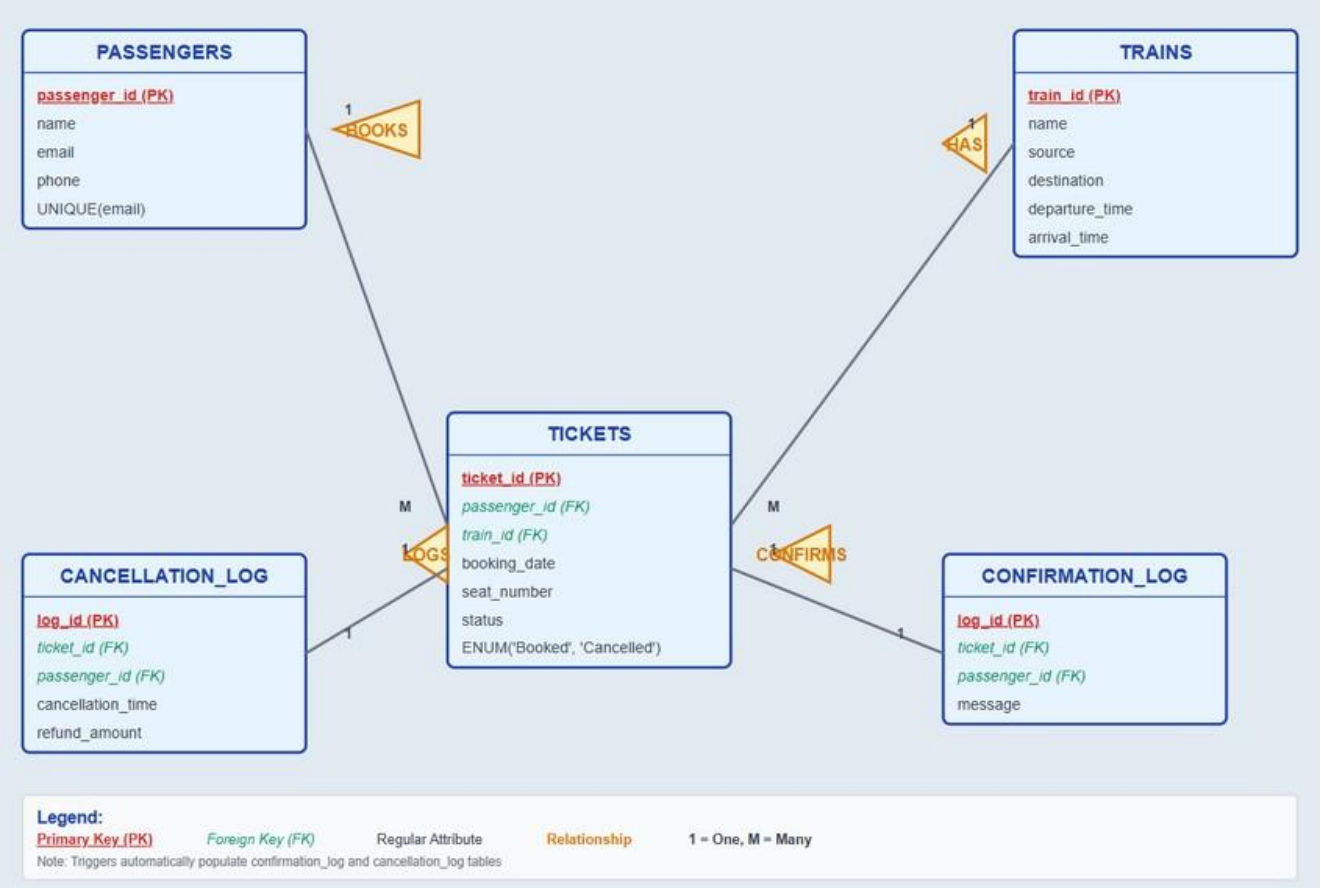


Figure . E R Diagram

This ER Diagram gives a brief idea about the relations existing between the tables and tells about the primary and the foreign keys being used in this Database.

SCHEMA DIAGRAM

A database schema can be represented in a visual diagram, which shows the database object and their relationship which represents the logical view of the database and how the relationships among them are represented.

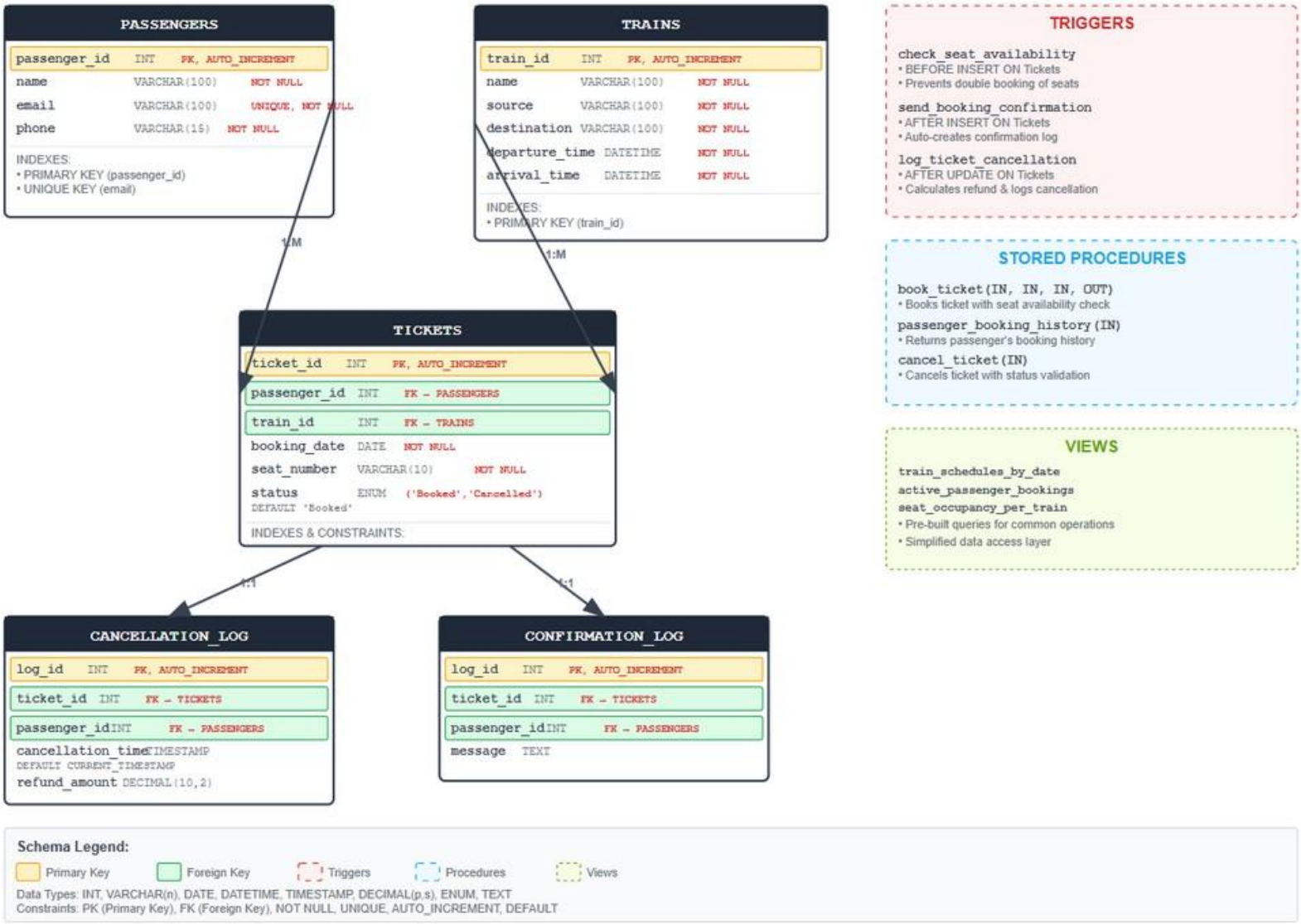


Figure :SCHEMA DIAGRAM

This Schema Diagram in Fig represents different tables used

TESTING QUERIES:

- SELECT * FROM Trains LIMIT 5;

Output:

←T→	train_id	name	source	destination	departure_time	arrival_time
Edit Copy Delete	6044	TVC chennai egmore special	secundrabad	villupuram	2025-07-08 20:39:08	2025-07-09 14:39:08
Edit Copy Delete	6059	CBE BJU special	MGR chennai CTL	secundrabad jn	2025-07-09 19:40:33	2025-07-10 08:40:33
Edit Copy Delete	6060	BJU CBE Express	secundrabad jn	MGR Chennai	2025-07-10 20:42:07	2025-07-11 10:00:00
Edit Copy Delete	7016	SC BBS special	kacheguda	srikakulam	2025-07-13 18:43:48	2025-07-14 10:00:00
Edit Copy Delete	7201	guntur jn secundrabad jn special	guntur jn	secundrabad jn	2025-07-01 06:25:48	2025-07-01 13:55:48

- SELECT * FROM seat_occupancy_per_train;

Output:

train_id	train_name	source	destination	booked_seats	cancelled_seats	total_tickets
6059	CBE BJU special	MGR chennai CTL	secundrabad jn	2	1	3
6060	BJU CBE Express	secundrabad jn	MGR Chennai	1	0	1
7016	SC BBS special	kacheguda	srikakulam	0	1	2
7238	BTT MENU SPECIAL	KURNOOL CITY	KAKINADA TOWN	0	1	1
7246	NS FESTIVL SPECIAL	RAICHUR	KAKINADA TOWN	0	1	1
12705	secundrabad jn intercity superfast	guntur	secundrabad jn	1	1	2
17201	Golconda express	guntur	secundrabad	1	1	2
17202	GOLCONDA EXPRESS	SECUNDERRABAD JN	GUNTUR JN	1	0	1
17228	guntur jn dhone express	guntur jn	dhone	0	2	2
17239	simhadri express	guntur	visakhapatanam	2	0	2
17243	guntur jn rayagada express	guntur	rayagada	0	2	2
17251	guntur jn kacheguda express	guntur jn	kachegunda	0	2	2
17252	KACHEGUDA GUNTUR JN EXPRESS	KACHEGUDA	GUNTUR JN	1	1	2
17277	DHONE GUNTUR JN EXPRESS	DHONE	GUNTUR JN	1	0	1
57306	guntur jn kachiguda passenger	guntur jn	kacheguda	2	0	2

- insert in to tickets values(null,5,6059,’2025-07-15’,’A2’,’Booked’);

Output:

```
insert into tickets values(null,5,6059,'2025-07-15','A2','Booked');
```

MySQL said:

```
#1644 - Seat already booked. Please choose another seat.
```

- CALL passenger_booking_history(5);

Output:

Extra options

ticket_id	train_name	source	destination	departure_time	arrival_time	booking_date	seat_number	status
31	CBE BJU special	MGR chennai CTL	secundrabad jn	2025-07-09 19:40:33	2025-07-10 08:40:33	2025-07-15	B3	Booked
3	BJU CBE Express	secundrabad jn	MGR Chennai	2025-07-10 20:42:07	2025-07-11 10:00:00	2025-06-30	B1	Booked

- CALL cancel_ticket(3);
  Output:

message
Ticket cancelled successfully.

- EXPLAIN select * from tickets where status='booking' and booking_date >'2025-07-01';

  Output:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	tickets	ALL	*NULL*	*NULL*	*NULL*	*NULL*	25	Using where

CONCLUSION

The Railway System Database project demonstrates how a structured relational database can effectively support a real-world railway reservation and management system. Key highlights of the project include:

- **Efficient Data Organization:** Core operations like managing train schedules, passenger information, and ticket bookings are handled using well-designed relational tables such as Trains, Passengers, and Tickets. Data is normalized to eliminate redundancy.
- **Dynamic Features via SQL:** The use of advanced SQL features such as triggers, views, and stored procedures makes the system automated and responsive. For example:
  Triggers ensure booking constraints and log events like cancellations.
  Views provide summarized insights such as occupancy reports.
  Stored procedures streamline tasks like booking or cancelling a ticket.
- **Analytical Capability:** The system includes analytical queries and reports to monitor:
  > Frequently traveled routes
  > High-volume passengers
  > Ticket booking trends and availability
- **Data Integrity and Optimization:** Proper use of foreign keys, constraints, and ENUMs ensures data accuracy. Indexing and optimized query design enhance performance and scalability.
- **User-Centric Design:** The database supports real-time tracking of bookings and cancellations. The inclusion of cancellation logs and refund policies makes the system practical and user-friendly.
- **Future Ready:** This backend database forms a robust foundation for future extensions, including:
  > Front-end web or mobile interfaces
  > Integration of payment gateways
  > Real-time train status and tracking

This project showcases the power of SQL in building smart, scalable, and maintainable reservation systems.